



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παράλληλα Συστήματα
Εργασία Mpi

Ονοματεπώνυμο: Ευάγγελος Τραϊκόγλου

Αριθμός Μητρώου: 1115202200189

- cpu: i5-6500
- cores: 4 physical
- os, kernel: Ubuntu 20.04.6 LTS, 5.4.0-216-generic
- comp version: gcc 9.4.0

Σημείωση: Εκτός της 3ης άσκησης, όλα τα πειράματα τρέξαν με ένα αρχείο όπου όριζε 4 διαθέσιμα slots (αφού τα linux διαθέτουν 4 πυρήνες ανά cpu και όχι hyperthreading) σε όλα τα μηχανήματα, εκτός του linux03 που δεν ήταν διαθέσιμο. Δηλαδή η εντολή ήταν:

Run: `mpirun -f machines -n <num of proc> ...`

Επιπλέον, μικρές αποκλίσεις που μπορεί να παρατηρούνται από το επιθυμητό αποτέλεσμα οφείλονται στη χρήση διαμοιραζόμενων υπολογιστικών πόρων. Αυτό αν φαίνεται, συνήθως πλήττει τα αποτελέσματα όπου το n είναι μεγάλο και εμπλέκονται πολλά μηχανήματα.

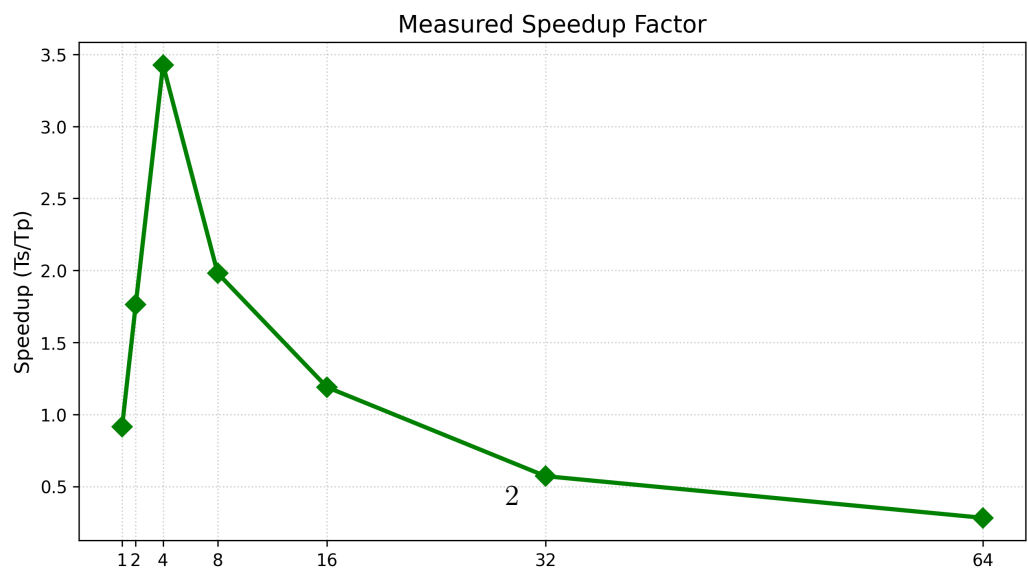
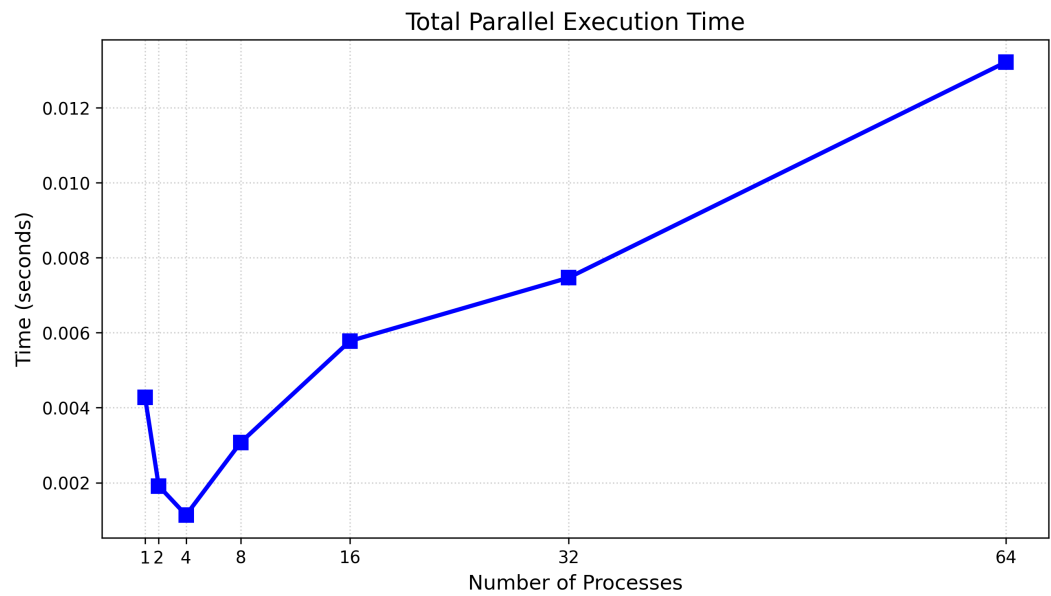
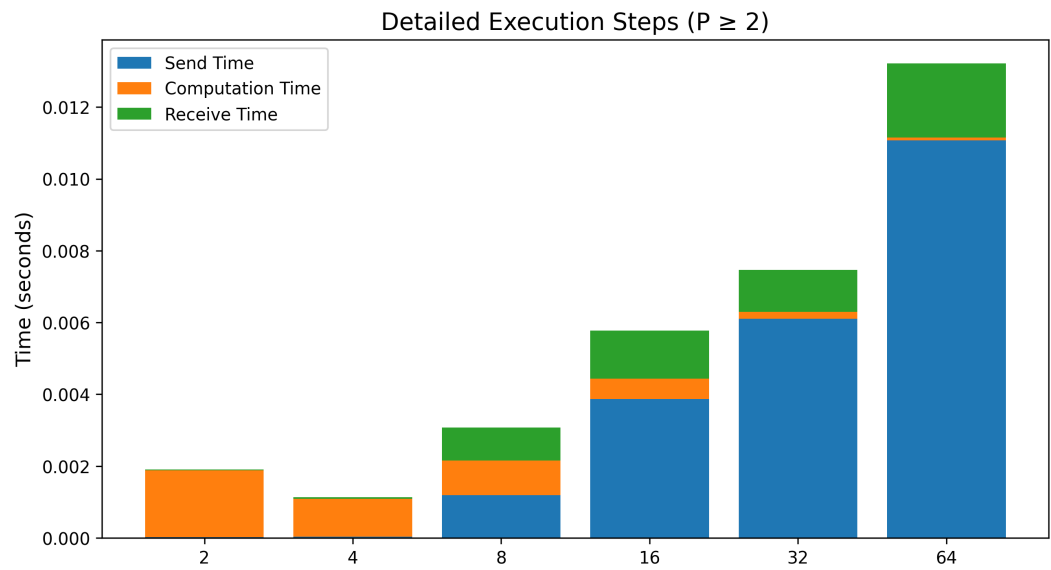
1 Polynomial Multiplication

Η πρώτη άσκηση επικεντρώνεται στον πολλαπλασιασμό πολυωνύμων βαθμού n . Ο σειριακός αλγόριθμος ακολουθεί την απλή υλοποίηση $O(n^2)$ χρόνου και τρέχει μόνο από την διεργασία 0. Στον παράλληλο, η διεργασία 0 αρχικοποιεί τα πολυώνυμα και τις δομές που χρειάζονται. Κάθε διεργασία υπολογίζει ένα μέρος του τελικού πολυωνύμου πολλαπλασιάζοντας ένα μέρος του `pol1` με ολόκληρο το `pol2`. Άρα το `pol2` γίνεται broadcast και το `pol1` Scatterv (καθώς το μέγεθος του `pol1` μπορεί να μη διαιρείται ακριβώς με το `num_proc`). Το αποτέλεσμα συγκεντρώνεται μέσω της Reduce στη διεργασία 0. Το local result array κρατά μερικό άθροισμα των συντελεστών του τελικού πολυωνύμου ($\text{rank} == 0 (A_0..A_k) \cdot (B_0..B_n)$, $\text{rank} == 1 (A_{k+1}..A_{2k}) \cdot (B_0..B_n)$, ...) και η Reduce παίρνει τα μερικά αθροίσματα σε κάθε θέση του local τελικού πολυωνύμου και τα αθροίζει και αποθηκεύει στη κατάλληλη θέση στο τελικό πολυώνυμο στην διεργασία 0.

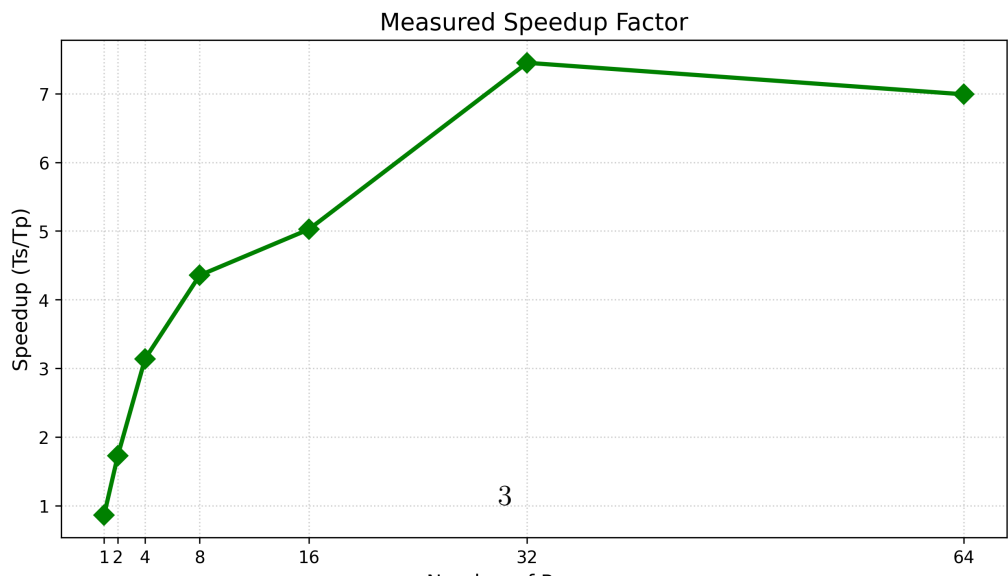
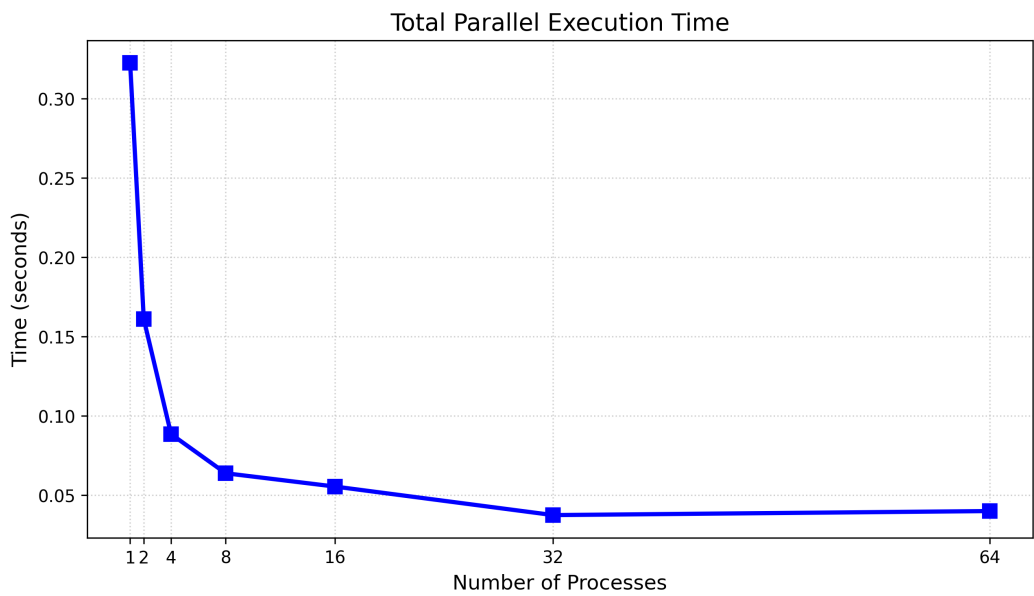
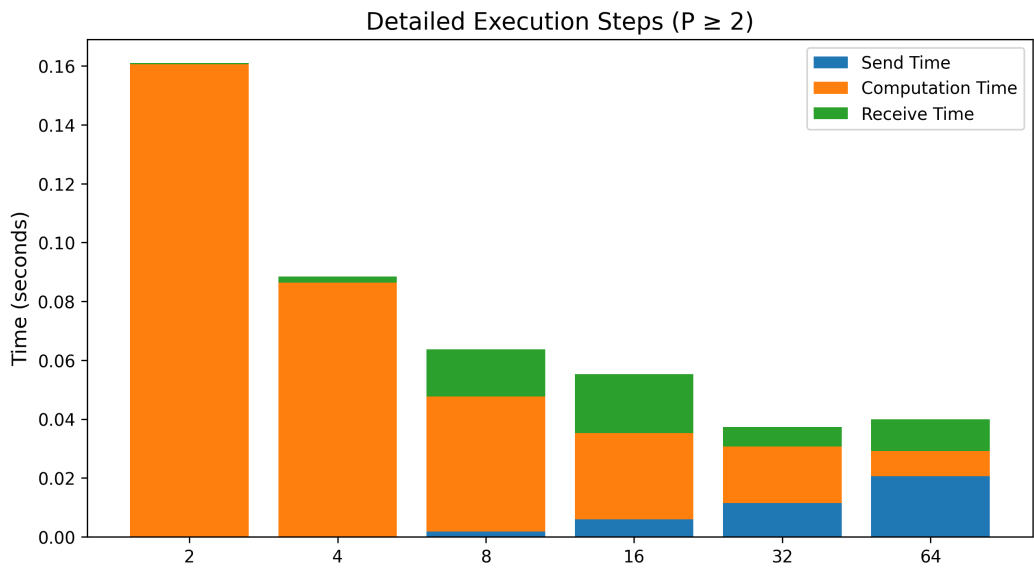
Run: `mpirun -n <n> ./polynomial_mult -n <degree>`

Σημείωση: Όλα τα πειράματα υπάρχουν στα csv αρχεία: `mpi_pol_results.csv`, `comparison_mpi_vs_omp.csv`.

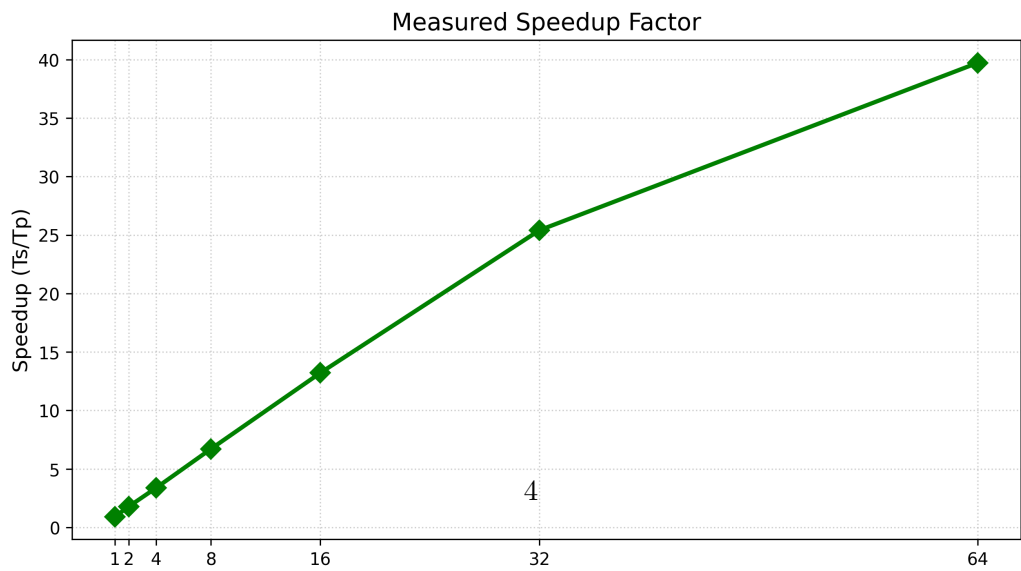
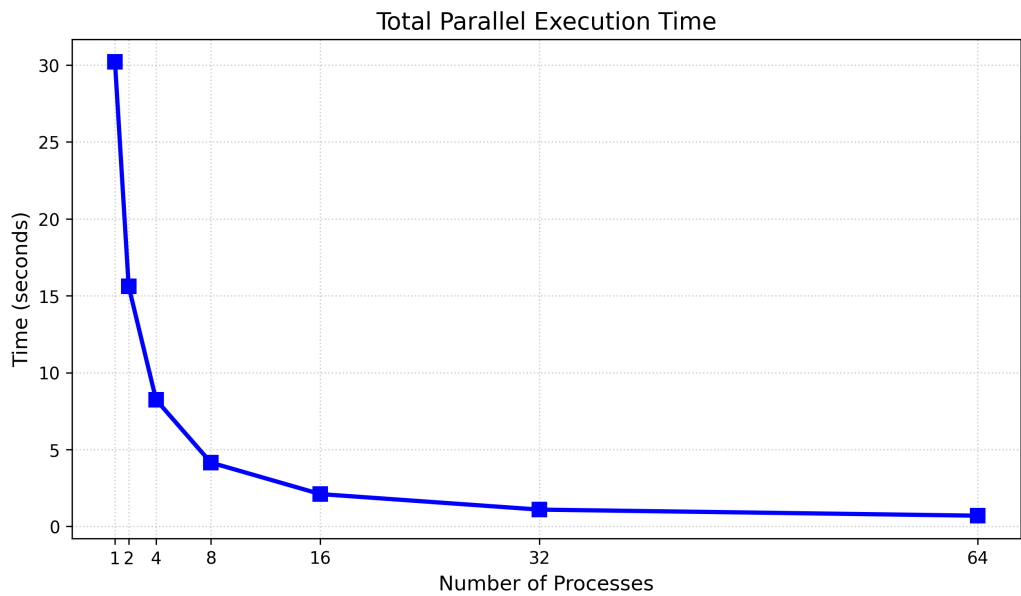
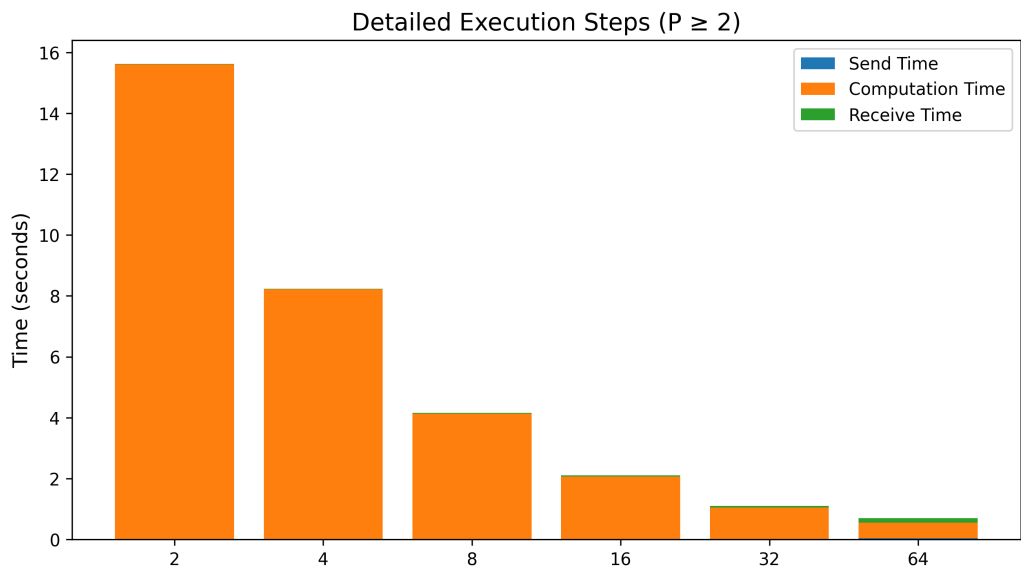
Performance Analysis: Polynomial Degree N = 1000

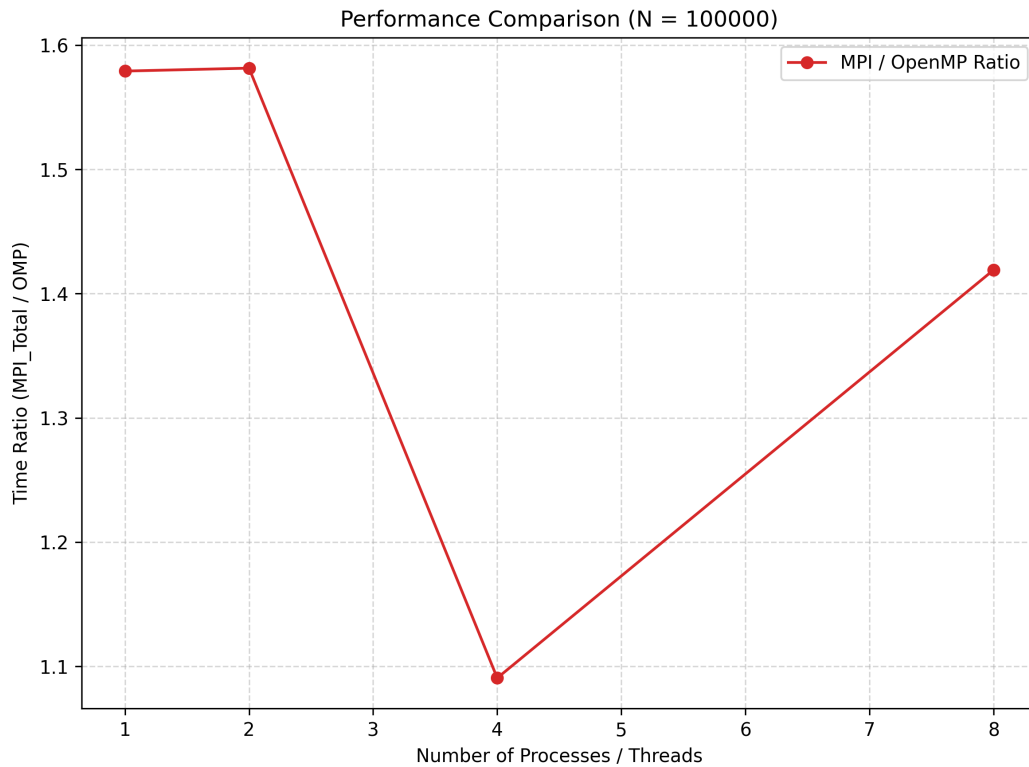


Performance Analysis: Polynomial Degree N = 10000



Performance Analysis: Polynomial Degree N = 100000





Συμπεράσματα: Για μικρά n και ειδικά όσο αυξάνεται ο αριθμός των διεργασιών ο περισσότερος χρόνος δαπανάται στην επικοινωνία παρά στο κομμάτι του υπολογισμού. Άρα και η επιτάχυνση έναντι του σειριακού είναι περιορισμένη με μέγιστο να φτάνει το ~ 3.5 . Βέβαια μετά από κάποιο σημείο η αύξηση του αριθμού των διεργασιών βλάπτει πετυχαίνοντας χειρότερους χρόνους, άρα και επιτάχυνση από τον σειριακό. Για μεγάλα n , ο χρόνος επικοινωνίας είναι αμελητέος μπροστά στο χρόνο υπολογισμού και η επιτάχυνση μεγαλώνει και μεγιστοποιείται στο ~ 40 σε σχέση με τον σειριακό για 64 διεργασίες. Η υλοποίηση με openmp είναι πιο αποδοτική της mpi σε έναν κόμβο με μέγιστη επιτάχυνση (έναντι της mpi) ~ 1.6 . Αυτό γιατί στην υλοποίηση με νήματα δε χρειάζεται κάποια επικοινωνία μεταξύ τους αφού λειτουργούν πάνω στα ίδια δεδομένα.

2 Sparse Matrix-Vector Multiplication

Η δεύτερη άσκηση επικεντρώνεται στον αποδοτικό πολλαπλασιασμό πίνακα-διανύσματος με χρήση της τεχνικής CSR. Η αρχικοποίηση γίνεται ως εξής: Κρατούνται τέσσερις πίνακες: values, col_index, row_index. Ο values κρατά τις μη μηδενικές τιμές του πίνακα με τη σειρά που διαβάζονται (ανά γραμμές). Ο col_index τις στήλες που αυτές οι τιμές βρίσκονται (με τη σειρά που εμφανίζονται στον values). Ο row_index πόσα στοιχεία είναι μη μηδενικά πάνω από τη γραμμή i (πχ $\text{row_index}[2] = \text{πόσα μη μηδενικά υπάρχουν πάνω από τη γραμμή } 2$). Οι πίνακες του CSR όπως και ο κανονικός δημιουργούνται στη διεργασία 0 σειριακά.

2.1 Dense Compute

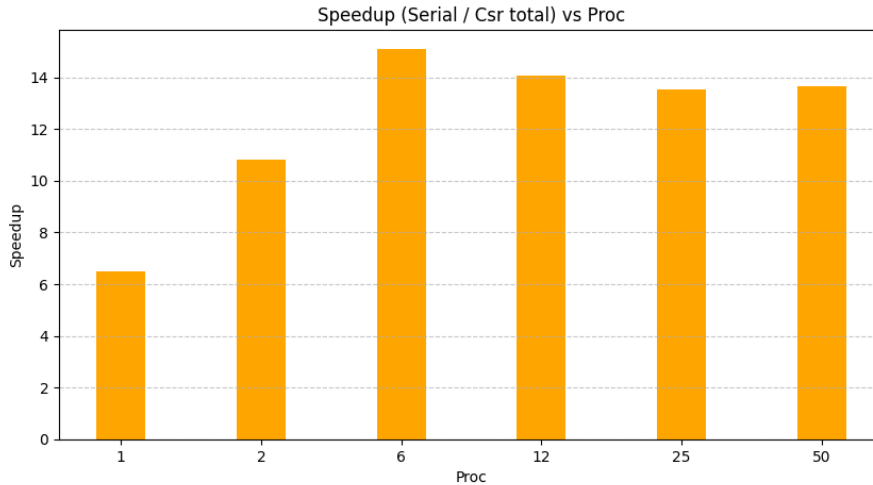
Ο κανονικός πίνακας σπάει ανά γραμμές. Επειδή μπορεί το n να μη διαιρείται με το πλήθος των διεργασιών (`num_proc`), χρησιμοποιούνται οι (`Scatterv`, `Gatherv`, `Allgatherv`) και υπολογίζονται οι πίνακες `sendcount`, `displacements`. Όπως και πριν το υπόλοιπο (σε πλήθος γραμμών) μοιράζεται στα πακέτα γραμμών που δημιουργούνται. Ο υπολογισμός είναι ο κλασσικός, μόνο που στο τέλος κάθε επανάληψης χρειάζεται το μερικό αποτέλεσμα που υπολογίζει κάθε διεργασία να πάει στις υπόλοιπες (`Allgatherv`). Στη τελευταία, τα μερικά αποτελέσματα συγκεντρώνονται στη 0. Ο χρόνος του παράλληλου υπολογισμού είναι το $\max=\{\text{execution time of proc } i\}$.

2.2 Csr Compute

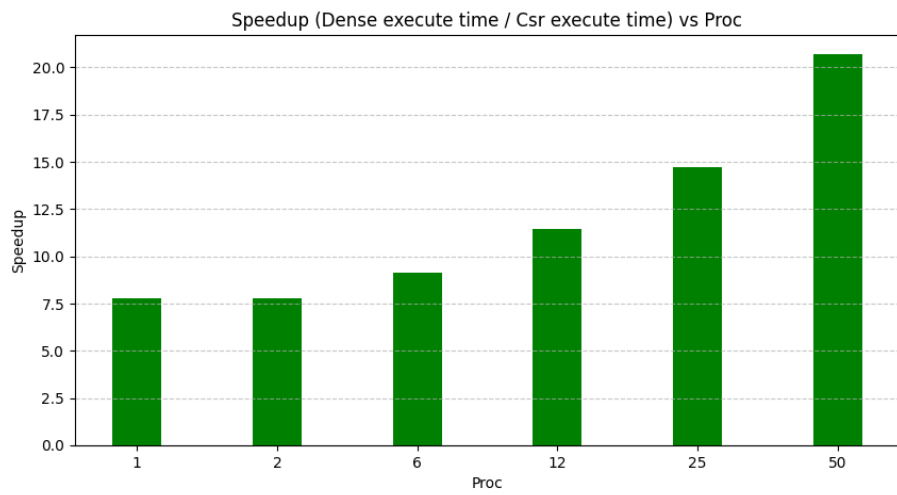
Μοιράζεται ο πίνακας `values` στις διεργασίες. Κάθε διεργασία παίρνει ένα πλήθος γραμμών που βρίσκεται πιο κοντά στον $\text{fair_size} = \text{non_zero}/\text{num_proc}$, δηλαδή αν βάζοντας και την επόμενη γραμμή του κανονικού πίνακα έχω $\geq \text{fair_size}$ τότε αυτές οι γραμμές (από το τελευταίο `split` μέχρι αυτή αποτελούν το πακέτο που θα πάει στη διεργασία i). Αυτό γίνεται για εξισορρόπηση φορτίου, επειδή η κατανομή των μηδενικών στις γραμμές δεν είναι γνωστή. Μάζι με τον πίνακα `values` μοιράζονται και οι `row/col.index`. Broadcast γίνονται οι `sendcount,displacements` πίνακες που χρειάζονται να συνοδεύουν το `scatter` που γίνεται στους πίνακες του `csr`. Η υλοποίηση του πολλαπλασιασμού είναι ακριβώς όπως περιγράφεται στο: <https://people.eecs.berkeley.edu/~aydin/csb2009.pdf>. Στο τέλος κάθε επανάληψης, χρησιμοποιείται όπως και στη `dense` περίπτωση `Gatherv,Allgatherv` ανάλογα με το αν είναι η τελευταία επανάληψη ή όχι. Ο χρόνος του παράλληλου υπολογισμού είναι το $\max=\{\text{execution time of proc } i\}$.

Run: `mpiexec -n <num_proc> ./sparse -n <matrix size> -p <sparsity:0-100> -i <iterations>`

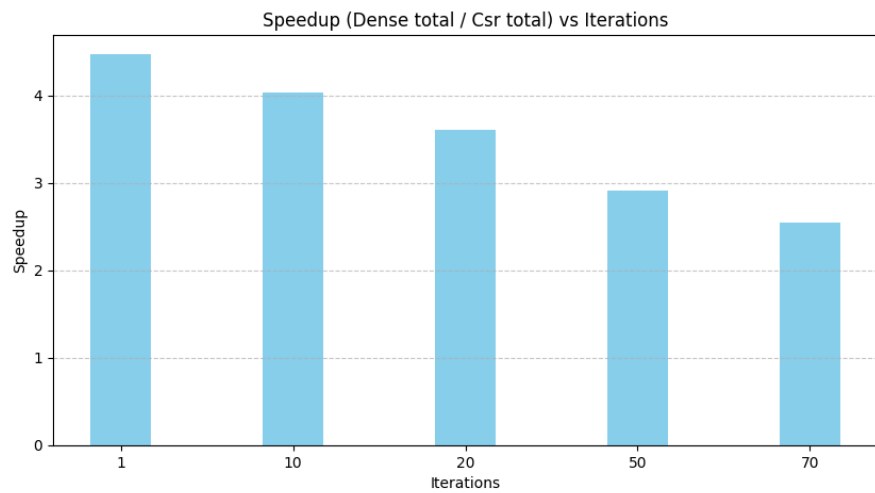
Σημείωση: Όλα τα πειράματα υπάρχουν και στο `csv` αρχείο: `benchmarks_sparse.csv`. Τα παρακάτω διαγράμματα προβάλλουν κάποια αποτελέσματα για δεδομένες τιμές των άλλων παραμέτρων (που επιλέχθηκαν για να αναδεικνύουν την υπεροχή του `CSR`).



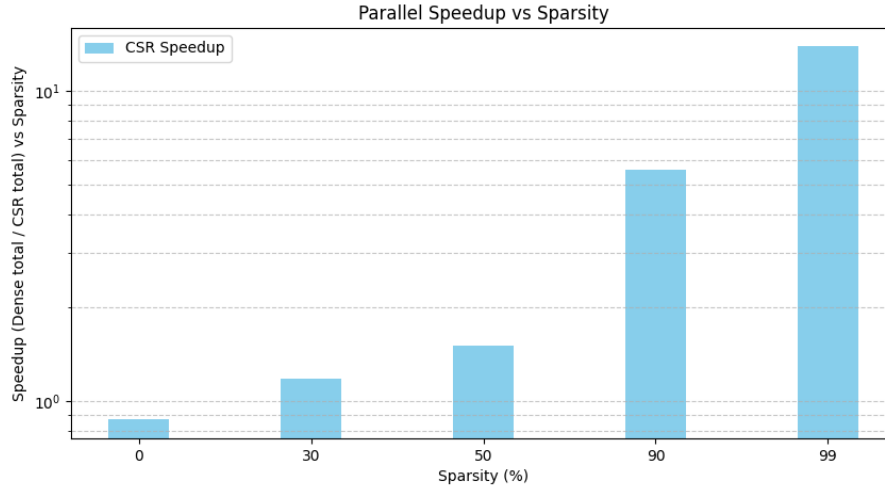
Σχήμα 1: Speedup vs procs: $n=10000$, $\text{sparsity}=90$, $\text{iter}=70$



Σχήμα 2: Speedup vs procs: n=10000, sparsity=90, iter=70



Σχήμα 3: Speedup vs iterations: n=10000, sparsity=90, proc=50



Σχήμα 4: Speedup vs sparsity: $n=10000$, $iter=70$, $proc=50$

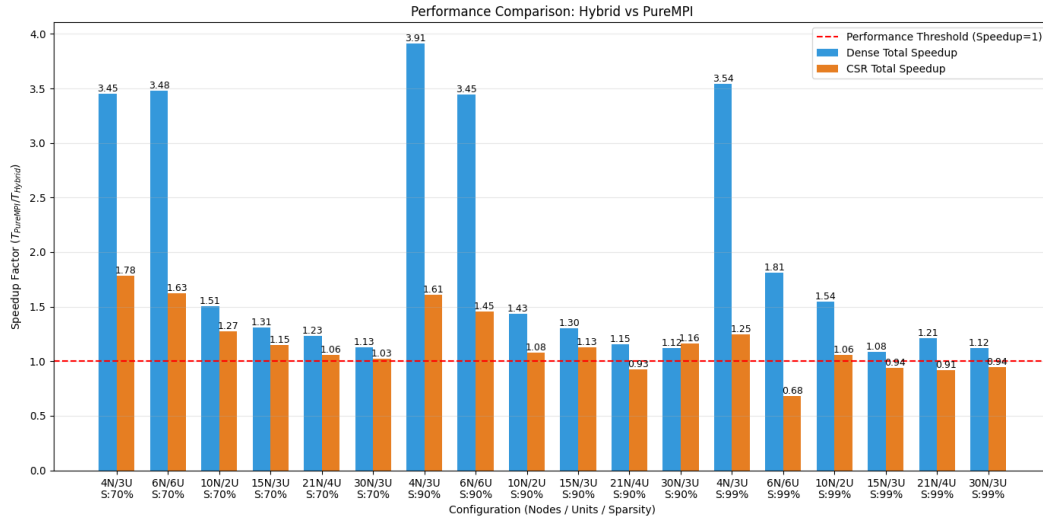
Συμπεράσματα: Η επιτάχυνση του CSR σε σχέση με τον σειριακό αλγόριθμο είναι μεγάλη σε υψηλά sparsity και μεγαλύτερους σε μέγεθος πίνακες. Η στασιμότητα στην επιτάχυνση (~ 14 για 12,25,50 διεργασίες) οφείλεται στον παραπάνω χρόνο επικοινωνίας που χρειάζεται καθώς αυξάνονται (επίσης όσα περισσότερα μνημάτα δεσμεύονται, τόσο μεγαλύτερη η πιθανότητα χρησιμοποιηθεί ένα ή περισσότερα που δεν είναι αδρανή). Καθώς αυξάνεται το sparsity και για ακούρντως μεγάλους πίνακες, το CSR υπερτερεί του απλού παράλληλου αλγορίθμου, αφού παραλείπονται οι ανούσιοι πολλαπλασιασμοί πετυχαίνοντας μέγιστη επιτάχυνση (έναντι αυτού) ~ 14 . Η αύξηση του αριθμού των επαναλήψεων μειώνει την επιτάχυνση του Csr έναντι του Dense (~ 4 στις 10, ~ 2.5 στις 70). Αυτό γιατί η επικοινωνία που χρειάζεται σε κάθε επανάληψη (gather), αρχίζει να γίνεται αισθητή καθώς αυξάνονται οι επαναλήψεις. Κοιτώντας μόνο το χρόνο υπολογισμού, η Csr υπερτερεί της Dense υλοποίησης και μάλιστα πετυχαίνοντας max επιτάχυνση: ~ 21 . Προφανώς αυτό παρατηρείται επειδή η Csr αποφεύγει μη αναγκαίες πράξεις.

3 Hybrid Sparse Matrix-Vector Multiplication

Η τρίτη άσκηση επεκτείνει τη 2η, προσθέτοντας την ικανότητα της χρήσης Openmp για τη δουλειά που έχει να κάνει κάθε διεργασία. Προστέθηκαν οι εντολές για openmp στο ίδιο αρχείο sparse_mat_vec.c με guards. Στις συναρτήσεις που υπολογίζεται ο πολλαπλασιασμός, δημιουργείται ένα parallel block και παραλληλοποιείται η εκτέλεση του πολλαπλασιασμού. Στο τέλος κάθε επανάληψης ένα από τα νήματα είναι υπεύθυνο να καλέσει τη allgather, gather.

Run: `mpirun -n <proc> ./hybrid_sparse -n <matrix size> -p <sparsity:0-100> -i <iter> -t <thr>`

Σημείωση: Τα πειράματα υπάρχουν και στο csv αρχείο: hybrid_comparison.csv.



Σχήμα 5: Speedup (mpi / hybrid), N=20000

Συμπεράσματα: Το κέρδος από την hybrid υλοποίηση είναι περιορισμένο (όπως φαίνεται και στο διάγραμμα). Υπάρχει μια βελτίωση ($\sim 3.5 - 4$) όταν χρησιμοποιούνται 4/6 κόμβοι, ενώ στις υπόλοιπες περιπτώσεις κινείται στους ίδιους χρόνους με την pure mpi υλοποίηση. Το αναμενόμενο ήταν να συμφέρει η Openmp γενικά, λόγω του μειωμένου συνολικού αριθμού διεργασιών σε σχέση με την pure mpi άρα και επικοινωνίας στη κάθε επανάληψη. Ίσως επιβαρύνει και το runtime της openmp.